A

PROJECT REPORT

On

PAHADNAMA TRAILS

Sahyadri Trek Discovery, Booking and Content Management Website

SUBMITTED BY

Vivek Amarjeet Chauhan | Roll No. 116

In partial fulfilment for the award of the degree

Of

BACHELOR OF COMPUTER SCIENCE

Semester V

Under the guidance of

Prof. Pritam Karanjkar Sir

Department of Computer Science

V. K. KRISHNA MENON COLLEGE OF COMMERCE & ECONOMICS

AND SHARAD SHANKAR DIGHE COLLEGE OF SCIENCE

Academic Year 2026-2027

CERTIFICATE

This is to certify that Vivek Amarjeet Chauhan, Roll No. 116, of Bachelor of Computer Science, Semester V, has completed the project entitled “Pahadnama Trails: Sahyadri Trek Discovery, Booking and Content Management Website” in partial fulfilment of the academic requirements for the year 2026-2027.

The project report records the design and implementation of a website for discovering Maharashtra Sahyadri treks, reviewing trail information, contacting the trek coordinator, and managing trek content through an administration console.

Date of Submission: _____

Project Guide Head / In-charge
Prof. Pritam Karanjkar Sir Department of Computer Science

College Seal Examiner

DECLARATION

I, Vivek Amarjeet Chauhan, Roll No. 116, declare that this project report, entitled “Pahadnama Trails: Sahyadri Trek Discovery, Booking and Content Management Website,” is submitted by me in partial fulfilment of the Bachelor of Computer Science programme, Semester V, for the academic year 2026-2027.

The report describes the project source code and supplied website materials to the best of my understanding. I have identified the external technical documentation used in the references. This work has not been submitted by me for another degree or award.

Place: Mumbai
Date: _____

Student Signature: _____
Vivek Amarjeet Chauhan

ACKNOWLEDGEMENT

I express my sincere gratitude to Prof. Pritam Karanjkar Sir, my project guide, for his guidance and support during the development and documentation of this project.

I thank the Department of Computer Science and V. K. Krishna Menon College of Commerce & Economics and Sharad Shankar Dighe College of Science for the academic setting in which I completed this work. I am also grateful to my family and friends for their encouragement.

Finally, I acknowledge the maintainers of the open-source tools and documentation used by the project, including Node.js, Express, SQLite and the browser standards that support the website.

TABLE OF CONTENT

This report is arranged from project context and requirements through system design, code implementation and the final operating workflow. The page numbers below refer to this report's 50-page edition.

Sr.No	Contents	Pg. No.
1	Project Proposal	6
2	Plagiarism Report	7
3	Introduction	8
4	Objectives	9
5	Scope	10
6	Methodology	11
7	Requirement Specification	12-14
8	Feasibility Study	15
9	Gantt Chart and Project Timeline	16
10	System Design	17-25
11	Code Implementation	26-38
12	Database Design and Relational Schema	39-41
13	Screenshots of the Application	42-45
14	Future Scope	46-47
15	Conclusion	48
16	Glossary	49
17	References	50

Project Proposal

Project title: Pahadnama Trails — Sahyadri Trek Discovery, Booking and Content Management Website. The project belongs to the areas of web engineering, database-backed information systems and tourism services.

The Sahyadri range contains forts, summits, waterfalls and forest routes that attract trekkers throughout the year. Planning a trip requires more than a destination name: visitors need to understand the route, difficulty, schedule, transport, food, equipment and safety expectations.

Pahadnama Trails brings curated Sahyadri trail information into one branded site. A visitor can compare trek options, inspect practical details and choose a public batch or enquire about a custom group trek. An administrator maintains trek records and related operational information through protected routes.

The central design objective is to reduce fragmented information during trek discovery. Instead of relying on separate pages or informal messages for every detail, the platform structures route, itinerary, pickup, pricing and FAQ information around each trek record. The browser interface is served by a Node.js server and backed by SQLite.

- Primary audience: people considering a Maharashtra weekend trek.
- Secondary audience: the Pahadnama team maintaining trails, dates, booking records and site settings.
- Deliverable: responsive public website, Express REST API, SQLite data store and administrative interface.

Plagiarism Report

No institutional similarity-check report was supplied for this project. A similarity percentage or plagiarism certificate is therefore not claimed in this report.

This page records the verification status honestly. Before submission, attach the original report generated by the college-authorized checker if the department requires one.

Verification item	Status
Similarity-check report	Not supplied
Similarity percentage	Not reported
Verification authority	College-authorized checker
Action before submission	Attach the original checker report if required

This is a status page, not a similarity certificate.

Introduction

Trek information can be scattered across social posts, chat messages and separate documents. That makes it harder for a visitor to compare options or confirm whether a route is suitable for their time, fitness and travel needs. The operating team also needs a consistent way to update content and batch availability.

The project addresses this coordination problem with a structured catalogue and a central content store. Each trek can carry a consistent set of attributes and related records. Public pages use these records to present route details, upcoming dates, gallery photos and questions. Admin routes provide the tools to maintain this information.

The platform does not replace local guides, forest authority instructions or direct confirmation from the organizer. It is an information and coordination layer; final logistics and any manual payment confirmation remain operational responsibilities.

- Information consistency across trek pages.
- Clearer discovery and comparison for weekend visitors.
- A manageable place for the team to update trail content and dates.
- An explicit contact path for booking questions and custom group requests.

Objectives

The project aims to provide a clear web experience for discovering Sahyadri treks and to support the basic content-management tasks behind that experience.

- Present a catalogue of Maharashtra treks with location, difficulty, duration, price and image.
- Show structured trek details, including itinerary, inclusions, exclusions, pickup points, equipment guidance and FAQs.
- Display available batch dates and their seat/status information when configured.
- Provide direct WhatsApp contact and a visible UPI booking explanation.
- Support feedback submission and administrator approval before public display.
- Provide protected administration endpoints for trek, date, photo, FAQ, booking, feedback and site-setting management.
- Store the operating records in SQLite using linked tables and foreign-key constraints.
- Keep the public site usable across common desktop and mobile viewport sizes.

Scope of the Project

The in-scope system includes the public trek catalogue and detail pages, the Express API, SQLite storage, selected media upload handling, the administration console and contact/booking-related flows. Trek records include descriptive, logistical and pricing information. Related records hold dates, photos, FAQs, reviews and bookings.

The source also includes Razorpay order creation and signature-verification routes. The visitor-facing page describes a manual UPI and WhatsApp process, while payment code includes simulated test behavior. This report treats those as separate paths and does not represent simulated payments as live transactions.

Included	Outside the current verified scope
Trek listing and details	Native Android/iOS application
Admin content and batch management	Automated seat locking under concurrent booking load
Feedback moderation and booking status records	Independent proof of a production payment deployment
WhatsApp enquiry links and UPI instructions	Third-party hotel inventory or live travel data
SQLite persistence and file uploads	Multi-region cloud scaling and analytics

Development Methodology

The project can be understood as an incremental software development effort. Requirements were organized around the visitor journey and administrator operations. The system was then divided into a public interface, API routes, database tables and uploaded media.

A practical implementation sequence is: establish the schema and sample records; build the public catalogue; add trek details and related data; connect visitor feedback and contact actions; implement administration and authentication; then refine responsive presentation and deployment instructions.

For academic review, each feature should be traced to a visible interface or API and its data fields. Code examples later in this report are selected from the supplied project files. The validation chapter lists cases that should be exercised before deployment; it does not claim that a formal automated test suite has been run.

Stage	Work product
Requirement analysis	Visitor and admin needs
Design	Route map, modules, data relationships
Implementation	Frontend, Express API, SQLite schema
Review	Manual flows and error-path checks
Deployment preparation	Runtime, dependencies, persistence and secrets

Functional Requirements

Functional requirements describe the actions supported by the application code.

The project assumes a Node.js installation that supports the built-in node:sqlite module, the project dependencies installed from package.json, and a writable directory for the SQLite database and uploads. The public and admin interfaces expect the Express server to host both static assets and API endpoints.

Trek information, dates, prices, route notes and contact settings are maintained by the operator. The software does not independently verify the accuracy of route conditions, safety claims, transport, meal arrangements or forest permissions. Visitors should follow current instructions from trek leaders and local authorities.

The current admin session registry is in server memory. Sessions are therefore tied to a running process. SQLite is appropriate for a compact deployment but requires an explicit backup and concurrency plan as usage grows. Upload validation and payment configuration also require production hardening.

- FR-01: The site shall show active treks in a public catalogue.
- FR-02: A visitor shall be able to request one trek by numeric identifier or slug.
- FR-03: Trek output shall include linked dates, photos, FAQs and approved reviews.
- FR-04: A visitor shall be able to submit feedback; new feedback shall be unapproved by default.
- FR-05: The application shall expose public contact and site settings while omitting the payment secret from the public settings response.
- FR-06: An administrator shall be able to log in and use protected management routes.
- FR-07: Admin functions shall manage treks, dates, photos, documents, FAQs, feedback, bookings and selected settings.
- FR-08: The application shall support an order-creation and payment-verification API path, including clearly separated simulated test behavior.
- FR-09: The public interface shall provide direct WhatsApp and UPI booking information.

Non-Functional Requirements

Non-functional requirements describe system qualities and constraints. The codebase provides a starting point; final conformance requires review in the deployment environment.

Quality	Requirement / consideration
Usability	Use clear labels for difficulty, time, price, itinerary and pickup information.
Responsiveness	Keep navigation, cards, modal forms and controls usable on phone and desktop widths.
Security	Use password hashing, protected admin APIs and safe secret handling; remove demo bypasses before production payments.
Reliability	Back up SQLite and uploaded files together; handle database and filesystem errors.
Maintainability	Keep server routes, database helpers, public assets and admin assets in identifiable files.
Performance	Avoid unnecessary large image transfers; compress and size photographic assets.
Compatibility	Use a Node runtime supporting node:sqlite and modern browser features used by the frontend.

Users and Use Cases

The platform has two primary user roles. A visitor browses information and contacts the organizer. An administrator signs in to maintain content and review operational records. The supplied source does not define customer accounts or a customer self-service portal.

- Visitor: browse treks; open details; inspect dates and FAQ; use contact links; submit feedback; view approved stories.
- Administrator: authenticate; review summary; create and update treks; manage batch dates and media; moderate feedback; inspect bookings; change settings.
- Trek coordinator: responds to enquiries and confirms practical details outside the website through the configured contact channel.

Feasibility Study

Technical feasibility: Node.js, Express, SQLite, browser HTML/CSS/JavaScript, cookie-parser and Multer provide a compact stack for a small content catalogue and administrative workflow. The existing project source contains the principal modules and sample trek data needed for a local demonstration.

Economic feasibility: the core runtime and database components are open-source. Hosting, domain registration, persistent disk and any payment provider fees depend on the chosen deployment and are not established by the source files.

Operational feasibility: the public interface presents a focused set of trek details. Operators can manage content through the admin console, but must protect credentials, maintain backups and review booking and payment states.

Schedule feasibility: the report does not assert a specific duration because no project timeline was supplied. The development phases can be fitted into the academic calendar by the student and guide.

Gantt Chart and Project Timeline

The source files do not record dated project milestones or actual phase durations. The table below presents a logical implementation sequence based on the delivered modules; it is not a historical schedule or a claim about time spent.

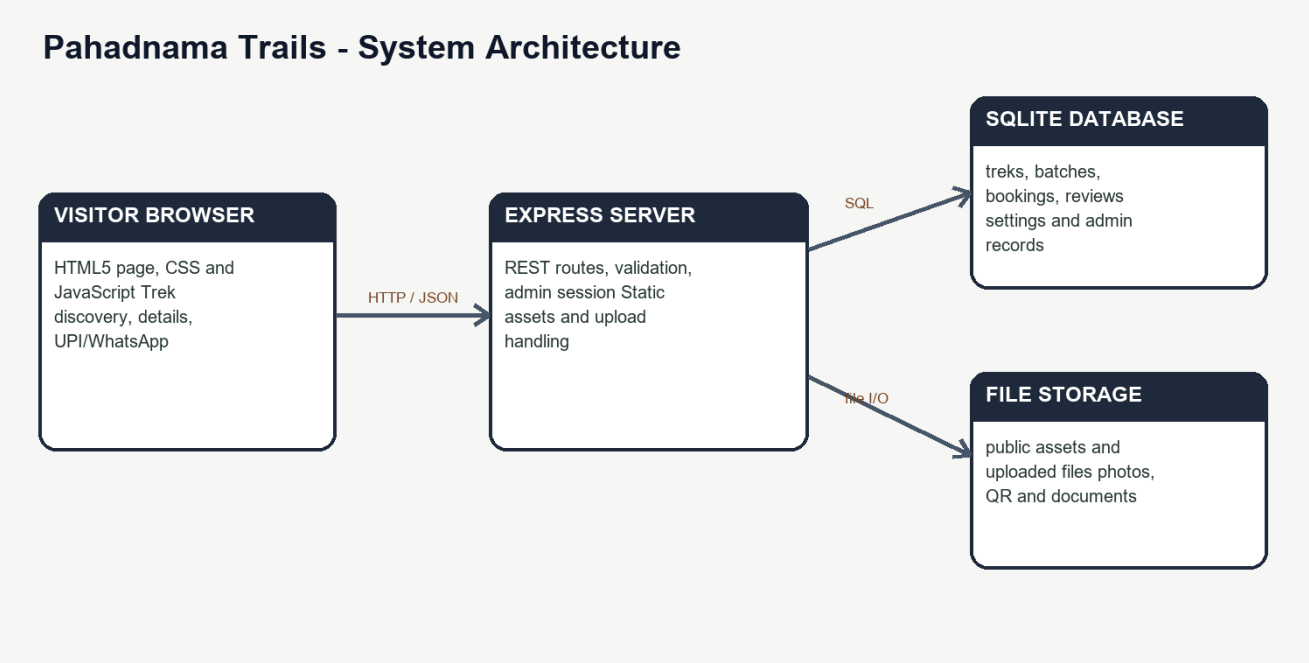
Project phase	Duration / dates	Work sequence
Planning and requirements	Not documented	Define visitor and administrator needs
System and database design	Not documented	Plan routes, API modules and relational data
Core implementation	Not documented	Build public pages, server routes and admin console
Content and integration	Not documented	Connect SQLite records, trek dates, media and feedback
Review and deployment	Not documented	Review site behavior and publish the Render deployment

Sequence summarized from project structure; actual dates and durations were not provided.

System Architecture

The application follows a compact client-server architecture. The browser loads HTML, CSS and JavaScript assets from Express. JavaScript requests JSON from public and administration endpoints. The server validates requests, executes prepared SQLite statements, processes uploaded files and returns JSON or static content.

SQLite persists application records in pahadnama.db. Images and documents are written to the uploads directory, while the public directory contains the application interface and brand assets. Optional payment API calls are made from the server so provider credentials are not sent as public browser settings.



Frontend Architecture

The public interface is implemented in vanilla HTML, CSS and ES6 JavaScript. The public page is rooted in public/index.html, visual rules live in public/style.css, and browser behavior is organized in public/app.js. The administrative page uses public/admin.html, public/admin.css and public/admin.js.

This division keeps the public experience and operations console separate while allowing both to call the same server. The frontend retrieves trek information from the API and uses DOM updates to show cards, details, booking dialogs, feedback and settings. Reusable functions support API calls, formatting, escaping and modal behavior.

The visual language focuses on outdoor photography, clear trail facts and an earthy brand presentation. Responsive CSS should preserve readable line lengths, touch-friendly controls and a predictable navigation pattern as the viewport narrows.

Backend and API Architecture

server.js creates the Express app, configures JSON and URL-encoded request parsing, cookie parsing and static file delivery. It defines public routes for treks, feedback, settings, FAQs and payments, as well as administration routes guarded by requireAdmin.

database.js opens SQLite with DatabaseSync, turns on foreign-key enforcement, creates tables and provides password hash helpers. Prepared statements are used for database operations. seed.js supplies sample content and a command-line seeding routine.

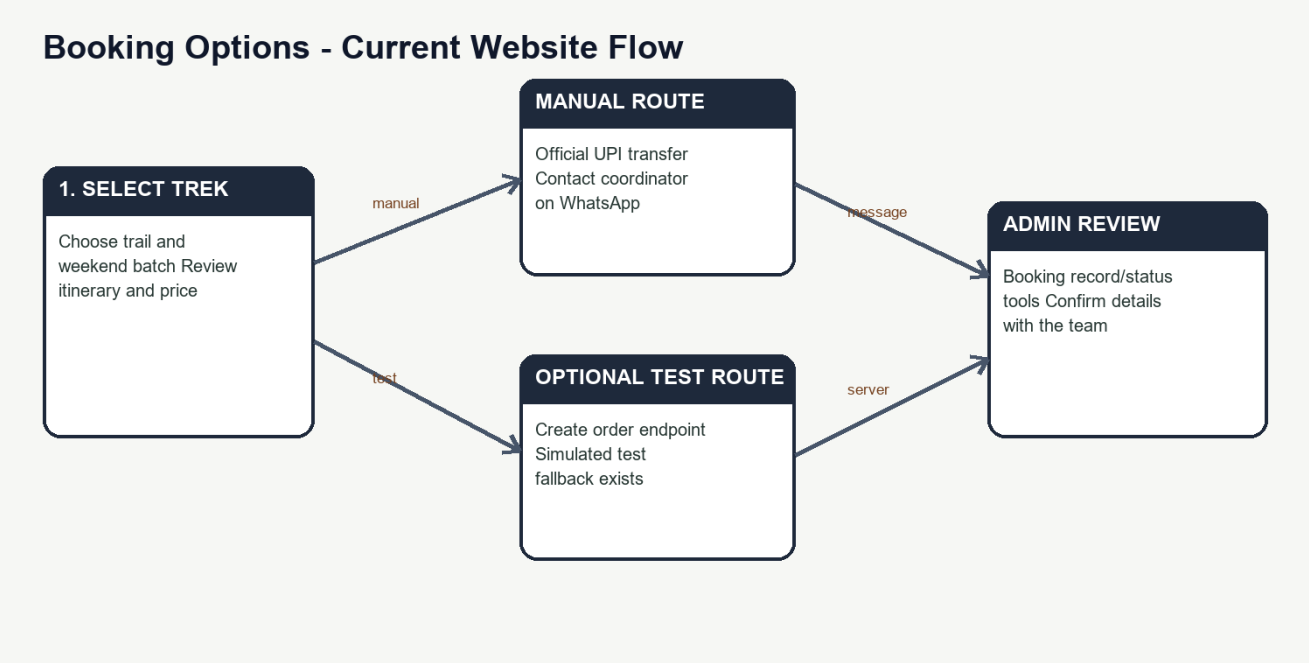
The current process keeps admin session tokens in a JavaScript Set. That is simple for a single-process demonstration, but it means a restart clears sessions and multiple processes do not share the same registry. Production deployment should use a persistent session store and appropriate cookie settings.

Visitor Booking Data Flow

A typical public data request begins in the browser and reaches an Express route. The route reads a trek or related records using parameterized SQLite statements, formats JSON fields and responds with a structured payload. The frontend then renders the result in the page.

For feedback, the browser sends name, rating, comment and an optional photo. The API validates the required text, stores the record with approved set to false, and returns a receipt. A moderator later changes the approval state through a protected admin endpoint. Only approved feedback is selected for public display.

The architecture separates the browser presentation, server-side rules, database records and media files. Error responses are returned when lookups or writes fail, allowing the client to display a recoverable message rather than treating every request as successful.



Public API Catalogue

Public API routes provide the data needed by the visitor interface. Responses are JSON unless the route serves a static asset.

Method and path	Purpose	Access
GET /api/treks	List active treks in featured order	Public
GET /api/treks/:id	Read a trek by numeric ID or slug	Public
GET /api/feedback	List approved feedback	Public
POST /api/feedback	Submit feedback and optional photo; starts pending approval	Public
GET /api/settings	Read public site settings; payment secret removed	Public
GET /api/payments/config	Read payment enabled flag, key ID and currency	Public
POST /api/payments/create-order	Create or simulate an order and pending booking	Public API
POST /api/payments/verify	Verify payment or accept test-mode fallback	Public API
GET /api/faqs	Read FAQ content	Public
POST /api/enquiries	Acknowledge an enquiry request	Public API

The /api/enquiries handler returns an acknowledgement response; the supplied implementation does not persist that enquiry in a database table.

Administration API Catalogue

Administration endpoints require an active session accepted by requireAdmin. They are used by public/admin.js to control the operations console.

Method and path	Function
POST /api/admin/login	Verify username/password and issue session cookie
GET /api/admin/me; POST /api/admin/logout	Check or clear current admin session
GET /api/admin/overview	Read counts, booking totals and settings summary
GET/POST /api/admin/treks	List or create trek records
PUT/DELETE /api/admin/treks/:id	Update or remove a trek
POST/DELETE /api/admin/treks/:id/photos	Upload or remove gallery photos
POST/PUT/DELETE /api/admin/treks/:id/dates	Manage batch dates and availability
GET/PUT/DELETE /api/admin/feedback/:id	Review and moderate feedback
GET/PUT /api/admin/bookings and settings	View booking records and update site configuration

Visitor Workflow - Discover a Trek

The home page introduces the Sahyadri weekend offering and links visitors to the trek catalogue. Cards give a quick comparison through route title, difficulty, duration, location, starting price and a short description.

Opening a trek detail view provides the deeper planning context: route highlights, itinerary, inclusions, exclusions, pickup locations, equipment, FAQ and relevant batch dates. The same structured data can be maintained by the team through the admin console.

The visitor workflow should answer three questions quickly: what will I do, what will I need, and how do I confirm details? Clear labels and direct links reduce the need to infer logistics from promotional copy.

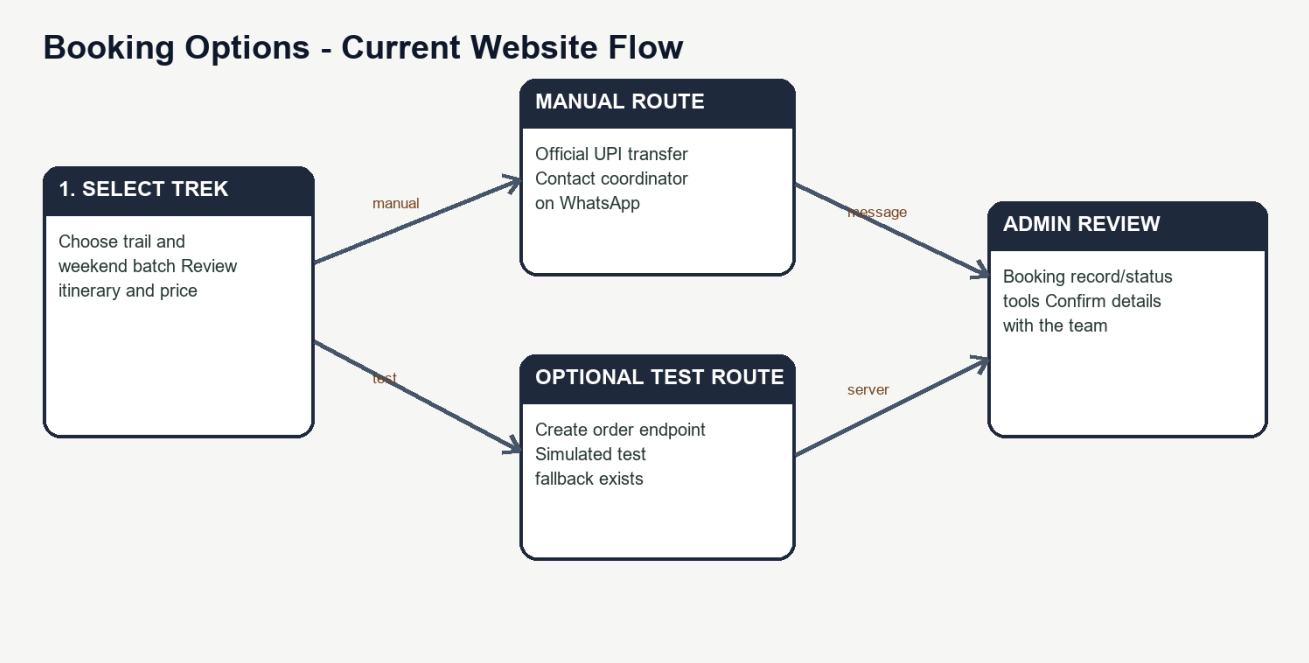
- Open the trek catalogue from navigation or the primary call to action.
- Compare routes by location, difficulty, duration and price.
- Open details and review schedule, pickups, gear and FAQs.
- Contact the coordinator or continue to the booking instructions.

Booking and Payment Flow

The current public page presents a manual booking option using UPI and WhatsApp. A visitor selects a trek, pays through the displayed UPI instructions, then contacts the coordinator with the payment reference and participant details. The organizer confirms the seat through the real operating workflow.

The source also contains a Razorpay order route. It validates customer fields and amount, creates a pending booking record, and calls the provider when configured. If credentials are placeholders or the provider call fails, the code can return a simulated test order. The browser and server contain test fallbacks, so a simulated “paid” response must never be treated as real money received.

For a real deployment, payment secrets must be managed outside source code, test success paths removed or isolated, signatures verified only against provider-issued values, and booking state reconciled with provider records.

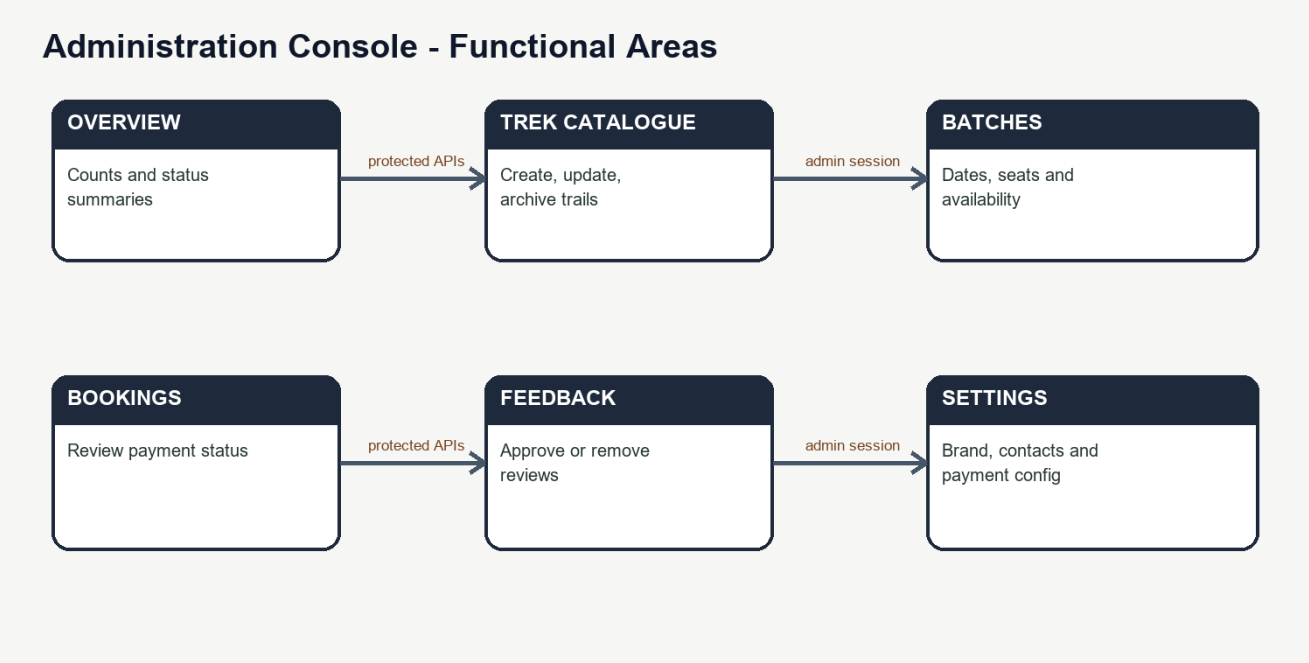


Administration Console

The administration console gives the operator a single workspace for catalogue and operational records. Its protected routes cover the dashboard overview, trek records, date batches, photos and documents, FAQs, customer feedback, bookings and selected site settings.

A role is not stored for public visitors because the supplied product does not define visitor accounts. Instead, the console verifies a single administrator identity and protects API operations with a server-side session token. Admin pages are separate static assets and are not a substitute for route-level authorization.

The overview endpoint reports counts of active treks, dates, pending and approved feedback, bookings and paid records. This is an operational snapshot, not a financial reconciliation report.



Security and Code Implementation Overview

The source hashes administrator passwords using Node’s scrypt function with a random salt and compares derived values using timingSafeEqual. Admin routes require a session token, and the login cookie is configured as HttpOnly and SameSite Lax. Public settings remove the payment secret before response.

Several deployment issues require attention. The session registry is in memory and the cookie is not marked Secure in the supplied code. The project includes development credentials and placeholder payment configuration; none are reproduced in this report. Replace all defaults, move secrets to environment or managed secret storage, use HTTPS, enable secure cookies, and remove simulated payment approval before taking live payments.

Customer names, ages, phone numbers and booking details are personal information. Limit access, avoid logging sensitive values, define retention and backup rules, and use the contact data only for trek coordination. Validate uploads by content as well as name and size, and constrain public upload exposure.

The code excerpts that follow are abridged from server.js, database.js, seed.js and the public application files. They omit credentials and unrelated markup. Prepared SQL statements bind user values separately from query text; the server returns status codes and JSON error objects for common failure paths.

Code Implementation - SQLite Setup

database.js opens the local SQLite file, enables foreign-key enforcement and creates the tables used by the website. Using CREATE TABLE IF NOT EXISTS makes startup able to initialize a fresh database while keeping an existing schema intact. Small ALTER TABLE blocks provide simple compatibility additions for fields introduced later.

```
const { DatabaseSync } = require('node:sqlite');
const path = require('node:path');
const db = new DatabaseSync(path.join(__dirname, 'pahadnama.db'));

// Keep related records consistent with their parent rows.
db.exec('PRAGMA foreign_keys = ON;');

db.exec(`
  CREATE TABLE IF NOT EXISTS treks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    slug TEXT UNIQUE NOT NULL,
    location TEXT NOT NULL,
    difficulty TEXT DEFAULT 'Moderate',
    price INTEGER DEFAULT 999,
    status TEXT DEFAULT 'active'
  );
`);
```

Source: database.js. The complete schema includes additional catalogue, schedule, media, feedback, booking, admin and settings fields.

Code Implementation - Password Hashing

The administrator password is not stored as plain text in the admins table. hashPassword derives a fixed-length value from the password and salt; verifyPassword repeats the derivation and compares bytes with a timing-safe comparison.

```
function hashPassword(password, salt = crypto.randomBytes(16).toString('hex')) {
  const hash = crypto.scryptSync(password, salt, 64).toString('hex');
  return { hash, salt };
}

function verifyPassword(password, hash, salt) {
  try {
    const check = crypto.scryptSync(password, salt, 64).toString('hex');
    return crypto.timingSafeEqual(
      Buffer.from(check, 'hex'), Buffer.from(hash, 'hex')
    );
  } catch {
    return false;
  }
}
```

Source: database.js. Seeded/default credentials still need to be replaced for any real deployment.

Code Implementation - Trek List Route

The public catalogue endpoint filters for active trek rows, places featured records first and formats each result before returning JSON. A try/catch boundary converts database errors into a server response rather than exposing a stack trace to the visitor.

```
app.get('/api/treks', (req, res) => {
  try {
    const rows = db.prepare(
      'SELECT * FROM treks WHERE status = ? ORDER BY is_featured DESC, id ASC'
    ).all('active');
    res.json(rows.map(formatTrek));
  } catch (err) {
    console.error('Error fetching treks:', err);
    res.status(500).json({ error: 'Failed to fetch treks' });
  }
});
```

Source: server.js. Values are passed to the prepared statement as parameters.

Code Implementation - Trek Formatter

A trek record contains JSON text for list-shaped fields. formatTrek parses those fields and attaches rows from dates, photos, FAQs and approved reviews. It also computes the next batch that is not marked full or cancelled.

```
function formatTrek(t) {
  if (!t) return null;
  const dates = db.prepare(
    'SELECT * FROM trek_dates WHERE trek_id = ? ORDER BY event_date ASC'
  ).all(t.id);
  const photos = db.prepare(
    'SELECT * FROM photos WHERE trek_id = ? ORDER BY id ASC'
  ).all(t.id);
  const upcoming = dates.find(d =>
    d.status !== 'FULL' && d.status !== 'CANCELLED'
  );
  return {
    ...t, highlights: parseJson(t.highlights),
    itinerary: parseJson(t.itinerary), dates, photos,
    next_date: upcoming ? upcoming.event_date : null
  };
}
```

Source: server.js. The supplied implementation also includes inclusions, exclusions, pickups, FAQs and approved reviews.

Code Implementation - Feedback Moderation

Feedback submission validates the visitor name and comment, clamps the rating to the supported range and stores the record with approved set to zero. This separates receipt of a review from its public publication.

```
const ratingNum = Math.min(5, Math.max(1, parseInt(rating) || 5));
const photoUrl = req.file ? `/uploads/${req.file.filename}` : '';

db.prepare(`
  INSERT INTO feedback
    (trek_id, name, trek_name, rating, comment, photo, approved)
  VALUES (?, ?, ?, ?, ?, ?, 0)
`).run(
  trek_id || null, name.trim(), trekName, ratingNum,
  comment.trim(), photoUrl
);

res.status(201).json({ ok: true, message: 'Feedback received' });
```

Source: server.js. Public feedback queries select approved records only.

Code Implementation - Create Order

The order endpoint accepts booking fields, checks required contact information and converts the total amount to paise. It creates a unique booking reference and inserts a pending booking record. Configured provider credentials can be used for an order request; placeholder credentials or a failed provider request lead to simulated test behavior in the source.

```
const amountInPaise = Math.round(amountVal * 100);
const uniqueSuffix = Date.now().toString(36).toUpperCase() +
  '-' + Math.floor(1000 + Math.random() * 9000);
const bookingCode = `PAHAD-${uniqueSuffix}`;

const insertStmt = db.prepare(`
  INSERT INTO bookings (booking_code, trek_id, trek_name,
    batch_date, customer_name, customer_phone, participants,
    total_amount, payment_method, payment_status)
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'razorpay', 'PENDING')
`);
```

Source: server.js. The full route validates a 10-digit phone, age range, positive amount and payment-enabled setting.

Code Implementation - Payment Verification

The configured verification path computes an HMAC-SHA256 digest for the provider order/payment pair and compares it with the submitted signature. The source also explicitly accepts test-prefixed IDs and a demo signature, so this code is not production-grade proof of a completed financial transaction.

```
const generatedSignature = crypto
  .createHmac('sha256', keySecret)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`)
  .digest('hex');

if (generatedSignature === razorpay_signature) {
  isValid = true;
}

// Development-only simulated payment branch exists below.
// Remove it and enforce provider verification before live use.
```

Source: server.js. The test bypass is a material limitation and must remain isolated from live payment processing.

Code Implementation - Admin Session

The login route reads the administrator row, verifies the submitted password with the database helper and creates a random session token. It writes an HttpOnly, SameSite cookie and stores the token in an in-memory Set. The requireAdmin middleware accepts that cookie or an explicit token header for API calls.

```
const sessionToken = crypto.randomBytes(32).toString('hex');
adminSessions.add(sessionToken);

res.cookie('admin_session', sessionToken, {
  httpOnly: true,
  sameSite: 'lax',
  maxAge: 7 * 24 * 60 * 60 * 1000
});

// Protected routes call requireAdmin before performing writes.
```

Source: server.js. The session Set is process-local; use a persistent store and Secure cookies behind HTTPS in production.

Code Implementation - Admin Content Route

The protected trek creation route builds a URL-friendly slug and inserts fields with a prepared SQL statement. Optional fields receive defaults. Similar route patterns update and delete trek records and manage related dates, photos and FAQs.

```
app.post('/api/admin/treks', requireAdmin, (req, res) => {
  const b = req.body;
  const slug = (b.name || 'trek').toLowerCase()
    .replace(/[^a-z0-9]+/g, '-')
    .replace(/(^-|-$)/g, '') + '-' + Date.now().toString().slice(-4);

  const stmt = db.prepare(`
    INSERT INTO treks (name, slug, location, difficulty, price, status)
    VALUES (?, ?, ?, ?, ?, ?)
  `);
  const result = stmt.run(
    b.name || 'Untitled Trek', slug, b.location || 'Maharashtra',
    b.difficulty || 'Moderate', parseInt(b.price) || 999, 'active'
  );
});
```

Source: server.js. The production route inserts the full trek field set and returns the created record.

Code Implementation - File Uploads

Multer handles images and documents uploaded through the admin interface. The image configuration limits file size and checks the filename extension and reported MIME type. A sanitized base filename plus a timestamp is used to reduce collisions.

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, UPLOADS_DIR),
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname).toLowerCase();
    const safeBase = path.basename(file.originalname, ext)
      .replace(/[^a-zA-Z0-9_-]/g, '-');
    cb(null, `${Date.now()}-${safeBase}${ext}`);
  }
});

const upload = multer({ storage, limits: { fileSize: 15 * 1024 * 1024 } });
```

Source: server.js. Production hardening should verify actual file contents and restrict how uploaded files are served.

Code Implementation - Frontend API Use

The browser uses fetch-based helpers to request JSON and then updates the page. An API boundary lets the public interface present a loading or error state separately from the database details. DOM output must continue escaping user-supplied names and comments before inserting markup.

```
async function loadTreks() {
  try {
    const response = await fetch('/api/treks');
    if (!response.ok) throw new Error('Unable to load treks');
    const treks = await response.json();
    renderTrekCards(treks);
  } catch (error) {
    showNotice(error.message);
  }
}
```

Representative pattern only. The project’s complete app.js contains the booking modal, payment flow, detail view and site interactions.

Code Implementation - Seed Data

seed.js provides sample trek rows and related itinerary data for local demonstration. It also clears selected content tables before inserting the seed collection. Therefore the seed command should be treated as a reset operation on its target database and should never be run against production data without an explicit backup.

```
const { db } = require('./database.js');

console.log('Seeding Pahadnama Trails database...');
db.exec('DELETE FROM photos;');
db.exec('DELETE FROM faqs;');
db.exec('DELETE FROM trek_dates;');
db.exec('DELETE FROM feedback;');
db.exec('DELETE FROM treks;');

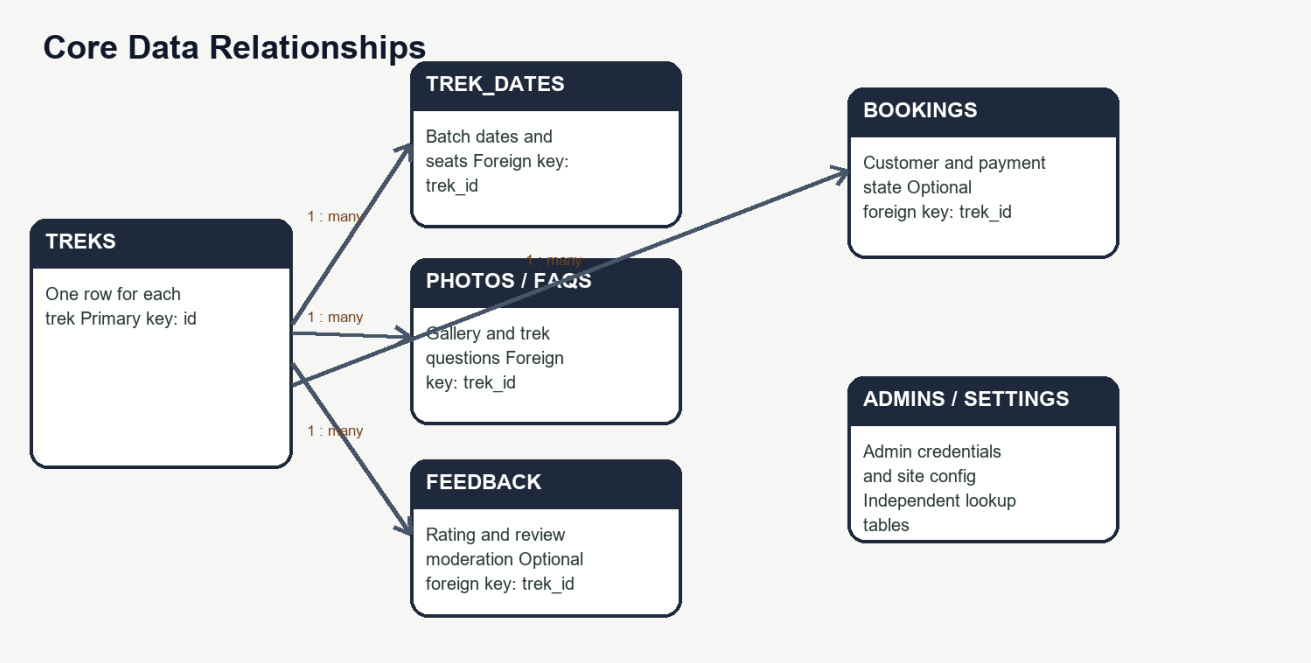
// Sample trek records are inserted below.
```

Source: seed.js. The sample data includes Sahyadri trails such as Harishchandragad, Kalsubai, Devkund and Rajgad.

Entity Relationship Diagram

The database centers on a trek record. A trek can have many dated batches, gallery photos, FAQ records, feedback entries and bookings. Related rows store the trek identifier as a foreign key. The schema enables cascading removal for date, photo and FAQ children, while booking and feedback references are set to null when a trek is removed.

Admin accounts and key/value settings are independent tables. A booking stores a snapshot of the trek name and customer details so that the booking record can still be read if its trek link is removed. The diagram shows the main logical connections; exact SQL constraints are defined in database.js.



Data Dictionary - Trek Catalogue

The treks table contains the public catalogue and the operational details for a route. JSON text columns store lists so a page can present highlights and a planned itinerary without adding one table per list item. IDs and slugs support stable references from API paths.

Field group	Example columns	Purpose
Identity	id, name, slug	Unique record and route-friendly lookup
Location	location, region	Base village, district or Sahyadri region
Route	difficulty, duration, distance, height, season	Planning and suitability facts
Commercial	price, original_price, is_featured, status	Displayed rate and catalogue visibility
Narrative	tagline, short_description, description	Summary and detail copy
Structured lists	highlights, itinerary, inclusions, exclusions, pickups, things_to_carry	JSON-encoded page sections
Operations	meeting_point, instructions, cancellation_policy	Practical visitor information
Media	cover_photo, document_url, document_name	Main image and optional document link

Data Dictionary - Operations Tables

Operational tables hold records connected to catalogue content, customer enquiries and website configuration.

Table	Key fields	Function
trek_dates	trek_id, event_date, available_seats, status	Weekend/date capacity and state
photos	trek_id, url, caption	Additional trek gallery assets
faqs	trek_id, question, answer, order_num	Questions displayed for a trek
feedback	trek_id, name, rating, comment, photo, approved	Review submission and moderation state
bookings	booking_code, trek_name, batch_date, customer fields, total_amount, payment_status	Booking request and payment references
admins	username, password_hash, salt	Administrator identity and password verifier
settings	key, value	Contact, theme and payment configuration

Responsive Design and Deployment

The public and admin stylesheets use CSS for layout, spacing, cards, dialogs, navigation and responsive behavior. The website includes branded imagery and hero content alongside trek-specific photographs. This gives the catalogue a visual identity while keeping route facts and booking instructions prominent.

Responsive behavior should be reviewed at narrow mobile widths, where long trek names, contact actions, date labels and detail accordions are most likely to wrap. Buttons should have clear focus states and adequate touch size. Image alternative text should explain the content where it is meaningful, and decorative backgrounds should not duplicate spoken content.

Theme settings are stored in the settings table and server-rendered into the root document to reduce a visible theme flash. The client also synchronizes selected values through local storage. User-controlled settings should be validated before being written into inline markup.

Local setup uses Node.js and the project dependencies. The package scripts expose a normal server start, a development watch mode and a seed operation. The server listens on process.env.PORT when supplied and otherwise uses port 3000.

The author reports deploying the website on Render at https://pahadnama-trails.onrender.com/. The public URL opened successfully during report preparation and displayed the Pahadnama Trails site title. This confirms the public page is reachable at that time; it does not verify database durability, payment configuration, or administrative security on the hosted service.

- Install a supported Node.js release with node:sqlite available.
- From the project folder, run npm install to install package dependencies.
- Run npm start to launch server.js; open the local site at the configured port.
- Use npm run seed only with a disposable development database because it deletes seeded-content tables before repopulating them.
- Render deployment: https://pahadnama-trails.onrender.com/; verify persistent storage for pahadnama.db and uploaded files and include them in a tested backup procedure.
- Set payment keys and admin secrets outside source code, serve through HTTPS, and configure Secure session cookies.

Do not publish development credentials, payment secrets, QR codes or customer data in the academic report or repository.

Website Content View - Trek Catalogue

The catalogue view uses a hero introduction and a group of trail cards. The supplied public page identifies Maharashtra Sahyadri weekend treks and presents route cards with location, difficulty, duration, price, availability and a short summary. A visitor can filter by groups such as beginner-friendly, moderate forts and summit-oriented routes.

The project uses trail photography alongside structured catalogue information.



Figure: Rajgad trail photograph from the project’s public uploads. Interface behavior is described from public/index.html and public/app.js.

Website Content View - Trek Details

A trek detail view expands the card-level summary into planning information. The stored fields support itinerary, highlights, inclusions, exclusions, meeting points, pickup options, what to carry, instructions and cancellation notes. Related photos, FAQs, ratings and batch dates can be attached without duplicating the main trek record.

The following image is one of the trek photographs supplied with the site assets.



Figure: Devkund water feature image supplied in the project assets. The web interface should present current conditions and operator instructions clearly.

Application Walkthrough

A visitor begins on the home page and chooses a trail from the catalogue. The detail panel supplies practical context and available date options. The visitor can then use the booking interface or contact link to confirm details with the coordinator.

A custom group of six or more can use the custom trek section to request a suitable date and pickup arrangement. The public contact section provides call, email, Instagram and direct WhatsApp entry points.

An administrator opens the separate admin page and signs in. Protected endpoints populate the dashboard, after which the operator may update a trek, adjust a batch, upload an image, respond to feedback moderation, inspect a booking or change selected website settings. Changes are reflected in public content through the same SQLite store.

- Home page → catalogue → trek details.
- Details → batch selection or enquiry.
- UPI / WhatsApp contact → organizer confirmation.
- Admin login → protected management APIs → SQLite and uploads.

Validation Plan

The following cases are a proposed manual validation checklist for academic review. They are test scenarios, not a claim that these cases have been executed or passed.

Area	Suggested case	Expected result
Catalogue	Request active trek list	Only active rows returned in featured order
Detail	Request unknown slug	404 response with not-found message
Feedback	Submit missing name/comment	400 response and no new row
Moderation	Submit valid feedback	201 response; approved flag remains false
Admin	Call protected route without session	401 response
Upload	Submit oversized or invalid image	Request rejected with a clear error
Payment	Use simulated test order	Clearly marked test flow; no real payment claim
Persistence	Restart service after content update	Database changes remain; admin memory session ends

Before external deployment, run the cases against a disposable database and review payment, file validation and session behavior.

Future Scope

Current limitations include process-local admin sessions, placeholder defaults in the source, simulated payment acceptance paths, a SQLite deployment model and an enquiry endpoint that does not persist requests. The included validation list has not been executed as part of this report. These limits should be addressed before expanding use or accepting online payments.

- Move all secrets to environment or managed secret storage; rotate defaults and separate development from production.
- Remove simulated payment success and reconcile real provider webhooks/orders before marking a booking paid.
- Use a persistent, shared session store and secure cookie settings behind HTTPS.
- Persist custom enquiries, add operator follow-up status and email/WhatsApp notification controls.
- Add automated tests for routes, schema migrations, input validation, moderation and failure paths.
- Add image resizing, content-type verification, upload access controls and backup/restore checks.
- If demand grows, assess a managed relational database and a durable media store.

Conclusion

Pahadnama Trails demonstrates a focused website for presenting Maharashtra Sahyadri treks and coordinating visitor enquiries. Its main strengths are the structured trek catalogue, detailed route information, linked batch data, operator-facing content management and a compact Node.js/SQLite architecture.

The implementation also reveals important deployment work: remove default secrets, isolate simulated payment paths, persist admin sessions, validate uploaded file contents and test data backup and restore. With these measures, the project can move from an academic demonstration toward a safer operational service.

Glossary

Term	Meaning
API	Application Programming Interface used by the browser and server
CRUD	Create, Read, Update and Delete operations
Foreign key	A field linking a row to a related table
HMAC	Keyed message authentication code used in signature calculation
Multer	Express middleware for multipart file uploads
REST	Resource-oriented HTTP interface pattern
SQLite	Embedded relational database engine
Trek slug	Readable unique route identifier for a trek
UPI	Unified Payments Interface used for direct account payments

References and Project File Map

Technical references are listed below. The project file map connects this report with the implementation.

Reference	Official source
Node.js Documentation - SQLite	https://nodejs.org/api/sqlite.html
Express 4.x API Reference	https://expressjs.com/en/4x/api.html
SQLite SQL Language	https://sqlite.org/lang.html
Razorpay - Server-side integration and signature verification	https://razorpay.com/docs/server-integration/
MDN Web Docs - HTML	https://developer.mozilla.org/docs/Web/HTML
MDN Web Docs - CSS	https://developer.mozilla.org/docs/Web/CSS
MDN Web Docs - Fetch API	https://developer.mozilla.org/docs/Web/API/Fetch_API
Multer Documentation	https://github.com/expressjs/multer
Live Render deployment	https://pahadnama-trails.onrender.com/
Project implementation	server.js, database.js, seed.js, package.json, public/index.html, public/style.css, public/app.js, public/admin.html, public/admin.css, public/admin.js

Academic year 2026-2027 | Project guide: Prof. Pritam Karanjkar Sir